Image Classification Using CNN with PyTorch

Alfido Tech AI/ML Internship — Task 2

Overview

This beginner-friendly project trains a Convolutional Neural Network (CNN) to classify the ten CIFAR-10 image categories. It includes dataset inspection, augmentation, training/validation curves, test metrics, a confusion matrix, model checkpointing, and a standalone inference script.

Results are not pre-filled. Run the notebook to produce actual metrics and figures. The trained file `best_cifar10_model.pth` is generated during training and is not included as a fake/untrained checkpoint.

Objective

Build, evaluate, save, and use a deep-learning image classifier with PyTorch.

Dataset

CIFAR-10 has 60,000 32×32 RGB images across airplane, automobile, bird, cat, deer, dog, frog, horse, ship, and truck. The standard split is 50,000 training images and 10,000 test images. This project separates 5,000 images from the 50,000-image training partition for validation; the test set is held out until final evaluation. Torchvision downloads the dataset on first run.

Technologies

Python 3, PyTorch, Torchvision, NumPy, Pandas, Matplotlib, scikit-learn, Seaborn, Pillow.

Installation

Use Python 3.10+ or Google Colab. In a virtual environment:

```
python -m venv .venv
# Windows: .venv\Scripts\activate
# macOS/Linux:
source .venv/bin/activate
python -m pip install --upgrade pip
pip install -r requirements.txt
```

For GPU support, install a PyTorch build compatible with your CUDA setup from the official PyTorch installation selector. Google Colab is the easiest beginner option.

Train

Open `deep_learning_cifar10.ipynb` in Google Colab or Jupyter and run cells from top to bottom. Dataset download requires internet access. The notebook selects the best checkpoint by validation accuracy and writes:

- `best_cifar10_model.pth`
- `dataset_samples.png`
- `training_curves.png`
- `confusion_matrix.png`
- `prediction_result.png`

- `training_history.csv`
- `evaluation_metrics.json`
- `classification_report.csv`

Evaluation methods

Final test metrics include accuracy, macro precision, macro recall, macro F1-score, per-class precision/recall/F1, and a confusion matrix. These values are computed from model predictions. The test set must not be used to tune the model or select a checkpoint.

Training curves

- **Loss:** lower values indicate fewer prediction errors under the loss function.
- **Accuracy:** higher values indicate more correctly classified examples.
- A large train/validation gap can indicate overfitting. Interpret curves alongside final test metrics.

Inference

After training has created `best_cifar10_model.pth`, run:

```
python inference.py path/to/your_image.jpg --model best_cifar10_model.pth
```

The script prints a predicted CIFAR-10 class and softmax confidence. Resize/normalization are applied automatically. Since CIFAR-10 contains small images, predictions on unrelated high-resolution photos may be unreliable.

Results

Fill this section only after running the notebook:

- Best validation accuracy: `see evaluation_metrics.json`
- Test accuracy: `see evaluation_metrics.json`
- Macro precision / recall / F1: `see evaluation_metrics.json`
- Attach the generated training curves, confusion matrix, sample grid, and prediction screenshot.

Transfer learning

The notebook contains an optional commented ResNet18 transfer-learning section. It is a separate experiment, not required for the baseline CNN. Enable it and evaluate using the validation split before final test evaluation.

Project structure

```
Deep_Learning_Image_Classification/
■■■ deep_learning_cifar10.ipynb
■■■ inference.py
■■■ requirements.txt
■■■ README.md
■■■ Project_Report.docx
■■■ screenshots/
    ■■■ README.txt
```

After training, add `best_cifar10_model.pth` and generated plots/metrics. Do not upload an untrained model and do not invent results.

GitHub submission

1. Create a new GitHub repository, e.g. `deep-learning-cifar10-cnn`.

2. Upload the notebook, `inference.py`, `requirements.txt`, `README.md`, report, and screenshots.

3. Run the notebook and add genuine output figures and measured metrics.

4. The `.pth` file may be too large or unnecessary for GitHub. If it exceeds the repository's 100 MB file limit, upload it to Google Drive or another approved storage provider and add a share link to this README. Set permissions so the evaluator can access it; test the link in a private/incognito window.

5. Commit and push, then share the repository URL with the internship coordinator.

References

- CIFAR-10: https://www.cs.toronto.edu/~kriz/cifar.html
- PyTorch tutorials: https://pytorch.org/tutorials/
- Torchvision datasets: https://pytorch.org/vision/stable/datasets.html
- scikit-learn metrics: https://scikit-learn.org/stable/modules/model_evaluation.html